Code

Stats 20, F23 – Homework 5

Aashna Sibal

11/14/2023

Question 1

```
char <- c("4", "2", "1", "0")
num <- 0:3
charnum <- data.frame(char, num, stringsAsFactors = TRUE)
```

(a)

```
as.numeric(char)
```

```
## [1] 4 2 1 0
```

```
as.numeric(charnum$char)
```

```
## [1] 4 3 2 1
```

In the first command, we are simply converting the character version of the numbers to the integer version, but the numbers themselves remain the same. In charnum, characters of vectors are coerced into factors due to the stringAsFactors argument being set to true. Therefore, the as.numeric gives the integer values of their levels, which end up being 4, 3, 2, and 1. This is why 0 is no longer present, since levels start at 1.

(b)

```
as.numeric(as.character(charnum$char))
```

```
## [1] 4 2 1 0
```

Question 2

(a)

```
simple_list <- list("vector" = 1:10, "matrix" = matrix(6:1, nrow = 3,
ncol = 2))
```

Reasons this does not work: 1) NULL is used to REMOVE objects, so you cannot add a NULL object. 2) Since NULL is also a built-in object, you need around it if you want to name a list component that.

(b)

```
simple_list[3] <- list(NULL)
length(simple_list) == 3
```

```
## [1] TRUE
```

(c)

```
simple_list$Vector <- NULL
simple_list$Vector
```

```
## NULL
```

Question 3

```
my_scale <- function(x) {
  z <- rep(0,length(x))
  index = 1

  if (is.null(attributes(x))) {
    for (i in x){
      z[index] <- ((i - min(x)) / (max(x) - min(x)))
      index <- index + 1
    }
  }
  else {
    a <- attributes(x)$a
    b <- attributes(x)$b
      for (i in x){
        z[index] <- a + (((i - min(x)) * (b - a)) / (max(x) - min(x)))
        index <- index + 1
       }
  }
  z
}
```

Question 4

(a)

Write a function factor_counter, that takes a factor, x, as an input and returns a list. In this list, the components' names should be the level names and the corresponding component should be the number of times the level appears. Include an argument, drop (default false), that determines whether or not the list should contain levels that are not used. Complete this without For example, if the input is factor(c(1,2,4,2)) with levels 1,2,3,4 and no drop argument defined, the result should be a list: "1" = 1, "2" = 2, "3" = 0, "4" = 1. Consider edge cases in which the input is NOT a factor, and you should give an error. Do not use the table() function.

Ideal Solution:

```
factor_counter <- function(x, drop = FALSE) {
  if (is.factor(x) == FALSE) {
    stop("Input is not a factor.")
  }
  if(drop) {
    x <- x[1:length(x), drop = TRUE]
  }

  result <- as.list(rep(0,length(levels(x))))
  names(result) <- levels(x)

  for (i in x) {
    name <- as.character(i)
    result[[name]] <- result[[name]] + 1
  }
 result
}
```

(b)

Consider the following function that takes a numeric vector, x, and counts how many values are in it. The input should be a vector, and an error should be raised if it is a matrix. Find 2 errors in it and write the proper code with the errors corrected.

```
my_function <- function(vec) {
  if (is.matrix(vec)) {
    for (v in vec[is.numeric(vec)]) {
      count <- count + 1
    }
   count
  }
  else {
    stop("Input is not a matrix.")
  }
}
```

Error 1: Count needs to be initialized before being added to. Otherwise, in the first loop count will not know what to add too and the function would not work. Once the count is initialized, the loop knows what to add it to. This is as simple as starting with 0, because that is a value that can be added to.

Error 2: The function is performing the count on vec if it is a matrix, meaning the function would essentially not work. The instructions say that we should raise an error if the input IS a matrix, so we would need to add a ! (not) operator. This is because we want to perform the function on vec if it is NOT a matrix specifically.

Working Solution:

```
my_function <- function(vec) {
  if (!is.matrix(vec)) { #fix 2
    count = 0 #fix 1
    for (v in vec[is.numeric(vec)]) {
      count <- count + 1
    }
   count
  }
  else {
    stop("Input is not a matrix.")
  }
}
```

```
my_function(c(1,2,3,4))
```

```
## [1] 4
```

(c)

(1)

Which of the below is a correct way to name components of a list? a) list(“name1” = 1:10,“name2” = 1:6) b) list_names = “name1”,“name2” c) names(list) <- c(“name1”,“name2”) d) list$1 = name1, list$2 = name2 e) A and C

E is the correct answer.

(2)

Which of the following commands will output an identity matrix?

a. diag(1)
b. diag(1, nrow = 1, ncol = 2)
c. diag(c(1,2,3))
d. diag(2)
e. diag(c(1,1,1), nrow = 4, ncol = 3)

D is the correct answer.

Question 5

(a)

```
my_unlist <- function(x) {
  result <- NULL
  class = lapply(x,class)
  highest_mode = max(sapply(x,mode))
  if (highest_mode == "character") {
    lapply(x[class(x) == "factor"], as.integer)
    lapply(x, as.character)
  }
  if (sum(any(class == "factor")) > 1) {
    new <- factor(NULL)
    levels(new) <- c(levels(x[class(x) == "factor"]))
    new_levels <- unique()
    print(levels(new))
    new <- factor(c(x[class(x) == "factor"]))
    result <- new
  }
  for (component in x) {
    result <- c(result, component)
  }

  result

}
```

(b)

```
stat_mode <- function(x, first = FALSE) {
  max <- 0
  x <- unlist(x)
  vals <- unique(x)
  result <- NULL
  for (i in vals) {
    if (length(x[x == i]) == max & first == FALSE){
      result <- c(result,i)
    }
    if (length(x[x == i]) > max) {
      max <- length(x[x == i])
      result <- i
    }
  }
  result
}
```

(c)

```
df_summary <- function(df) {
  column_data = list()
  alpha <- sort(colnames(df))
  index = 1
  for (i in alpha) {

    current <- unlist(df[[i]])
    if (class(current) == "character") {
      column_data[[i]] <- list("class" = class(current),
                               "modes" = stat_mode(current),
                               "mode_count" = length(current[current
== stat_mode(current)])
                               )
    }
    if (class(current) == "numeric") {
      column_data[[i]] <- list("class" = class(current),
                "min" = min(current, na.rm = TRUE),
                "mean" = mean(current, na.rm = TRUE),
                "max" = max(current, na.rm = TRUE),
                "na_count" = sum(is.na(current))
                )
    }

  }
  list("n_obs" = nrow(df),
       "n_var" = ncol(df),
       "var_names" = colnames(df),
       "column_data" = column_data)
}
```

(d)

```
load("starwars.RData")
results <- df_summary(starwars)
```

```
## Warning in current == stat_mode(current): longer object length is
not a
## multiple of shorter object length

## Warning in current == stat_mode(current): longer object length is
not a
## multiple of shorter object length
```

(e)

```
results$column_data$starships$modes
```

```
## [1] "X-wing"            "Millennium Falcon"
```